OOP in C++

Complete Chapter Notes

Chapter 1: Introduction to OOPs | Chapter 2: Classes, Objects & Access Specifiers

Chapter 1 — Introduction to OOPs

1.1 Definition of OOP

Object-Oriented Programming (OOP) is basically a programming style used in modern programming. It primarily revolves around classes and objects.

OOP refers to a language that uses the concept of class and object in programming.

The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Popular OOP Languages: C++, Java, Python, PHP, C#

Main Objective of OOP: To implement real-world entities such as polymorphism, inheritance, encapsulation, abstraction, etc.

1.2 The 4 Main Features / Pillars of OOP

These four features make OOP languages different from non-OOP languages:

- **Abstraction —** The property by virtue of which only the essential details are displayed to the user. Unnecessary details are hidden.
- **Inheritance —** Allows you to create class hierarchies where a base class gives its behavior and attributes to a derived class.
- **Polymorphism —** Ensures that the proper method will execute based on the calling object's type.
- **Encapsulation —** Allows you to control access to your object's state while making it easier to maintain or change your implementation at a later date.

1.3 Class (Overview)

A class is a logical entity used to define a new data type. It is a user-defined type that describes what a particular kind of object will look like.

A class is a template or blueprint for an object. A class contains variables, methods, and constructors.

Classes are very useful in programming. For example, if you need 100 cars, you can use the same car class to create 100 objects of type 'car'. The basic structure is the same; each just has its own name and properties.

The car class would allow the programmer to store similar information unique to each car (different models, colors, etc.) and associate the appropriate information with each car.

Syntax to Define a Class

```
class class name {
    // class body
    // properties
    // methods
};
```

Key points about the syntax:

- **class —** keyword used to create a class in C++.
- **class_name —** the name of the class.
- **class body —** enclosed in curly braces { }.
- **Semicolon (;) —** required after the closing curly brace.

1.4 Object (Overview)

An object is an instance of a Class. It is an identifiable entity with some characteristics and behavior.

Objects are the basic units of object-oriented programming. It may be any real-world object like a person, chair, table, pen, animal, car, etc.

A simple example: a car object has a model number (property) and can start or move (methods).

Code in OOP is organized around objects. Once you have objects, they can interact with each other to make something happen.

You need a class before you can create an object. When a class is defined, no memory is allocated. Memory is allocated only when an object is created (instantiated).

Syntax to Create an Object in C++

```
class_name objectName;                      // stack allocation
class_name * objectName = new class_name(); // dynamic / heap allocation
```

- **objectName —** the name of the object created by class_name.
- In dynamic allocation, the default constructor is called; memory is allocated on the heap and its address is assigned to the pointer objectName.

1.5 Real-World Class Modeling — Animal Example

Let's take Animal as a class and Mammals, Birds, and Amphibians as its objects.

Every animal is different, but they share common properties. The Animal class defines properties shared by all animals; each object holds its own specific values.

Full Code — Animal Class with Objects

```
#include <iostream>
using namespace std;

// creating Animal class
class Animal {
    bool gives_birth;
    bool lay_egg;
    bool live_in_ground;
    bool live_in_water;
    bool have wings;
};

int main() {
    // creating an object of animal class
    Animal mammal;
    mammal.gives_birth   = true;
    mammal.lay_egg       = false;
    mammal.live_in_ground = true;
    mammal.live_in_water  = false;
    mammal.have_wings    = false;

    Animal amphibian;
    amphibian.gives_birth   = false;
    amphibian.lay_egg       = true;
```

```
    amphibian.live_in_ground = true;
    amphibian.live_in_water  = true;
    amphibian.have_wings    = false;

    Animal bird;
    bird.gives_birth   = false;
    bird.lay_egg       = true;
    bird.live_in_ground = true;
    bird.live_in_water  = false;
    bird.have_wings    = true;
}
```

The Animal class provides a template or blueprint for creating objects (mammal, bird, and amphibian). Each object holds different values for the same properties.

1.6 Why Do We Need OOP?

- To make the development and maintenance of projects more effortless.
- To provide the feature of data hiding that is good for security concerns.
- We can solve real-world problems if we are using object-oriented programming.
- It ensures code reusability.
- It lets us write generic code: which will work with a range of data, so we don't have to write basic stuff over and over again.

1.7 Industry Example — OOP Using Cars

Consider needing not one car but 100 cars. Rather than describing each from scratch, you use the same Car class to create 100 car objects.

Each car object (Ferrari, BMW, Mercedes) will have its own: Year of Manufacture, Model, Top Speed, Color, Engine Power, Efficiency, etc.

The car class allows the programmer to store information unique to each car (different models, colors, top speeds) and associate the appropriate information with each one.

Object Values from the Industry Example

	Mercedes	Ferrari	BMW
year_of_manufacture	2021	2019	2020
model	Vision AVTR	Roma	BMW S6
top_speed	80 mph	198 mph	155 mph
color	Greyish-Silver	Red	Blue
Methods	start(), speed_up(), speed_down(), stop()	start(), speed_up(), speed_down(), stop()	start(), speed_up(), speed_down(), stop()

Chapter 2 — Classes, Objects & Access Specifiers

2.1 Class — Detailed

A class is a logical entity used to define a new data type. It is a user-defined type that describes what a particular kind of object will look like. A class contains variables (data members), methods, and constructors.

Class is a blueprint or a set of instructions to build a specific type of object. It is a fundamental concept of OOP which revolves around real-life entities. Class determines how an object will behave and what it will contain.

Data encapsulation is supported with 'class'. The class consists of both data and functions. The data in a class is called a member; functions in the class are called methods.

The 3 Components of a Class

- **Data Members —** Variables declared inside a class using fundamental data types (int, char, float, etc.) or derived data types (class, structure, pointer, etc.).
- **Methods —** Equivalent of a function in OOP. They are the actions that perform operations. A method accepts parameters as arguments, manipulates them, and produces output when called on an object.
- **Constructor —** Special class function that performs initialization of every object. In C++, it is automatically called when an object is created. It has no return type and has the same name as the class itself.

Syntax

```
class class_name {
    // data members
    // constructor (optional)
    // methods
};
```

- **class —** keyword to create a class in C++.
- **class_name —** the name of the class.
- **class body —** curly braces surround the class body.
- **Semicolon (;) —** required after the closing curly brace.

Complete Smartphone Class Example

```
class smartphone {
    // Data Members (Properties)
    string model;
    int    year_of_manufacture;
    bool   _5g_supported;

    // Constructor
    smartphone(string mod, int manu, bool _5g_supp) {
        model                = mod;
        year_of_manufacture  = manu;
         5g supported        =  5g supp;
    }

    // Methods
    void print_details() {
        cout << "Model : "               << model                << endl;
        cout << "Year of Manufacture : " << year_of_manufacture << endl;
        cout << "5g Supported : "        << _5g_supported       << endl;
    }
};
```

2.2 Object — Detailed

An object is an instance of a Class. It is an identifiable entity with some characteristics and behavior. To access the members defined inside the class, we need to create an object of that class.

Objects are the basic units of OOP. It may be any real-world object like a person, chair, table, pen, animal, car, etc.

Code in OOP is organized around objects. Once you have objects, they can interact with each other to make something happen.

Object — Smartphone Example with Full Code

A smartphone has a model number (property) and can send SMS (method). Using the smartphone class to create three objects:

```
#include <iostream>
using namespace std;

class smartphone {
    string model;
    int    year_of_manufacture;
    bool   _5g_supported;

    smartphone(string model_string, int manufacture, bool _5g_) {
        model               = model_string;
        year_of_manufacture = manufacture;
        _5g_supported       = _5g_;
    }

    void print_details() {
        cout << "Model : "               << model               << endl;
        cout << "Year of Manufacture : " << year_of_manufacture << endl;
        cout << "5g Supported : "        << _5g_supported       << endl;
    }
};

int main() {
    // Creating objects of smartphone class
    smartphone iphone  ("iphone 11",        2019, false);
    smartphone redmi   ("redmi note 11 t",   2021, true );
    smartphone oneplus ("oneplus nord",       2020, true );

    // Accessing class variables using dot syntax
    int    iphone_manufacture_date = iphone.year_of_manufacture;
    bool   redmi_support_5g        = redmi._5g_supported;
    string oneplus_model           = oneplus.model;

    // Calling methods on objects
    iphone.print_details();
    redmi.print_details();
    oneplus.print details();
}
```

Key Rules for Using Objects

- ★ To create an object, specify the class name, followed by the object name.
- ★ **To access class attributes / data members,** use the dot syntax (.) on the object followed by the attribute name.
- ★ **To call any method,** use the dot syntax (.) on the object followed by the method name.

MEMORY RULE:
• When a class is DEFINED → no memory is allocated.
• When an OBJECT is CREATED → memory is allocated.

You need to have a class before you can create an object.
When a class is instantiated (object created), memory is allocated.

Graphical Representation — Smartphone Class & Its Objects

The smartphone class is the parent blueprint. Three objects (Iphone, Redmi, OnePlus) are created from it, each holding their own property values:

smartphone (class)	Iphone (object)	Redmi (object)	OnePlus (object)
year_of_manufacture	2019	2021	2020
model	iphone 11	redmi note 11 T	oneplus nord
_5g_supported	False	True	True
Method: print_details()	print_details()	print_details()	print_details()

2.3 Access Specifiers

Access Specifiers in a class are used to assign access to the class members. They set restrictions on class members from being accessed by outside functions directly.

Access specifiers have a vital role in securing data from unauthorized access. They allow us to determine which class members are accessible to other classes and functions and which are not.

There are three types of access modifiers available in C++:

- Public
- Private
- Protected

1. Public

All the class members with a public modifier can be accessed from anywhere — inside and outside the class.

```
class person {
public:
    string name;   // accessible from anywhere
};
```

2. Private

All the class members with a private modifier can ONLY be accessed by the member function inside the class. They cannot be accessed from outside or by child classes.

```
class person {
private:
    int fb password;   // only accessible inside the class
};
```

3. Protected

The access level of a protected modifier is within the class and outside the class through a child class (or subclass). If you do not make a child class, it cannot be accessed outside the class.

```
class person {
protected:
    string assets;   // accessible in class + child/derived classes
};
```

4. Default (No Specifier)

By default, in C++, ALL class members are PRIVATE if you don't specify an access specifier.

```
class person {
    int name;   // by default, it is a PRIVATE data member
};
```

Complete Smartphone Class with All Access Specifiers

```
class smartphone {
    // Data members
    string model;               // by default private

public:
    int year_of_manufacture;    // public data member

protected:
    string company_name;        // protected data member

private:
    int password;              // private data member

    // Methods
private:
    void unlock_lockscreen() { // private method
    }

public:
    void call() {              // public method
    }

protected:
    void about_phone() {       // protected method
    }
};
```

Access Specifier Scope Table

Specifier	Inside Class	Child (Sub-class)	Outside Class
Public	✓	✓	✓
Protected	✓	✓	✗
Private	✓	✗	✗

Chapter 3 — Interview Questions

Q1. Why do we use OOPs?

- It gives clarity in programming and allows simplicity in solving complex problems.
- Data and code are bound together by encapsulation.
- Code can be reused, and it reduces redundancy.
- It also helps to hide unnecessary details with the help of Data Abstraction.
- Problems can be divided into subparts.
- It increases the readability, understandability, and maintainability of the code.

Q2. Differences Between Constructor and Method

Constructor	Method
It is a block of code that initializes a newly created object.	It is a group of statements that can be called at any point in the program using its name to perform a specific task.
It has the same name as the class name.	It should have a different name than the class name.
It has no return type.	It needs a valid return type if it returns a value; otherwise void.
It is called implicitly at the time of object creation.	It is called explicitly by the programmer by making a method call.
If a constructor is not present, a default constructor is provided by Java.	In the case of a method, no default method is provided.

Q3. What Are the Main Features of OOPs?

- Inheritance
- Encapsulation
- Polymorphism
- Data Abstraction

Q4. Disadvantages of OOPs

- Requires pre-work and proper planning.
- In certain scenarios, programs can consume a large amount of memory.
- Not suitable for a small problem.
- Proper documentation is required for later use.

Q5. Difference Between Class and Structure

- **Class —** User-defined blueprint from which objects are created. It consists of methods or sets of instructions that are to be performed on the objects.
- **Structure —** A structure is basically a user-defined collection of variables of different data types.

Q6. Difference Between a Class and an Object

Class	Object
Class is the blueprint of an object. It is used to create objects.	An object is an instance of the class.
No memory is allocated when a class is declared.	Memory is allocated as soon as an object is created.
A class is a group of similar objects.	An object is a real-world entity such as a book, car, etc.
Class is a logical entity.	An object is a physical entity.
A class can only be declared once.	Objects can be created many times as per requirement.
An example of class can be a car.	Objects of the class car can be BMW, Mercedes, Ferrari, etc.

Quick Reference — Cheat Sheet

Object Creation

```
ClassName obj;                      // stack allocation (normal)
ClassName *obj = new ClassName();   // heap allocation (dynamic)
```

Dot Syntax — Accessing Members

```
obj.dataMember;        // access a data member
obj.method();          // call a method
ptr->dataMember;       // via pointer (dynamic allocation)
```

Complete Class Template

```
class MyClass {
    int privateVar;              // private by default

public:
    int publicVar;

    MyClass(int a, int b) {      // constructor — same name, no return type
        privateVar = a;
        publicVar  = b;
    }

    void show() {               // method
        cout << privateVar << publicVar;
    }
};
```

Access Specifier Quick Reference

Specifier	Inside Class	Child (Sub-class)	Outside Class
Public	✓	✓	✓
Protected	✓	✓	✗

Private / Default	✓	✗	✗

Key OOP Terms at a Glance

Term	Meaning
Class	Blueprint / template for creating objects.
Object	Instance of a class; real memory-allocated entity.
Data Member	Variable declared inside a class.
Method	Function declared inside a class.
Constructor	Special method called automatically at object creation; same name as class, no return type.
Encapsulation	Binding data and functions together; hiding internal details.
Abstraction	Showing only essential details to the user.
Inheritance	Child class inherits properties and behavior from parent class.
Polymorphism	Same function name behaves differently based on calling object's type.
Access Specifier	public / private / protected — controls visibility of class members.
Default Access	In C++, members without a specifier are PRIVATE by default.